



Dependency-Aware Code Smell Refactoring via Multi-Agent LLM System

Havriil Pietukhin, Ivan Polášek



Financované
Európskou úniou
NextGenerationEU

PLÁN [OBNOVY]

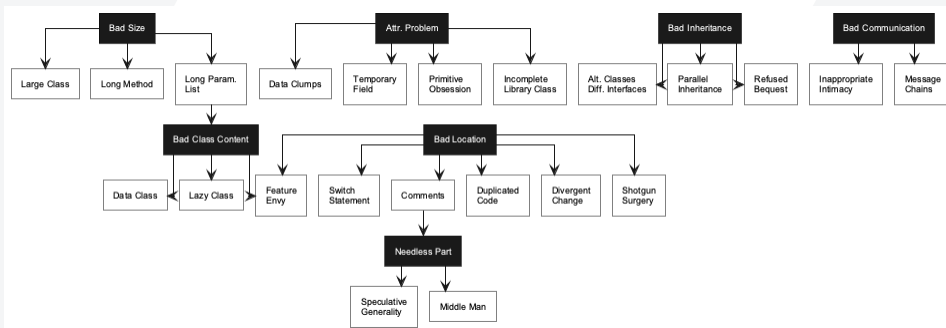


ÚRAD PODPREDESEDU VLÁDY
SLOVENSKEJ REPUBLIKY
PRE PLÁN OBNOVY
A ZNALOSTNÚ EKONOMIKU



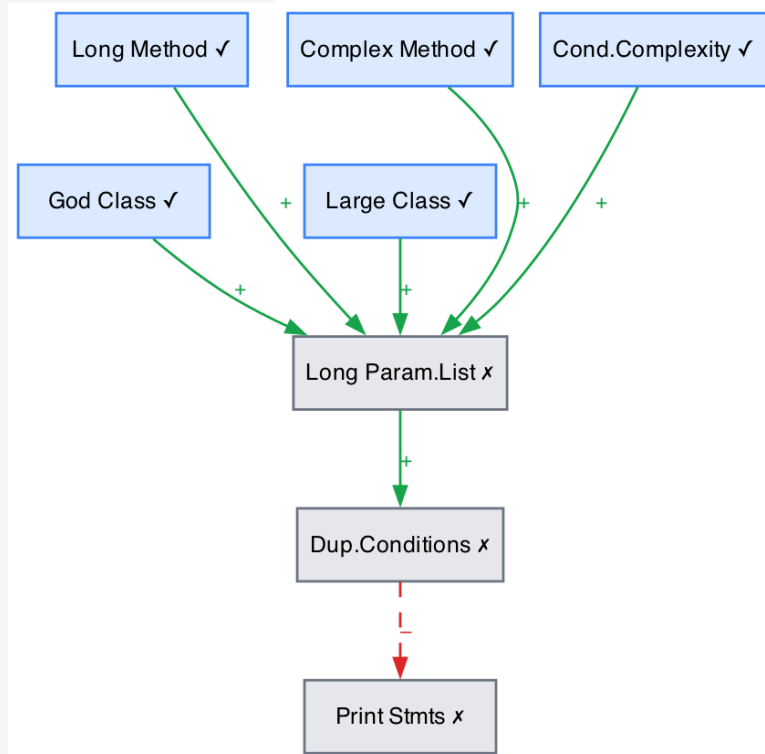
VÝSKUMNÁ
A INOVAČNÁ
AUTORITA

Research goal



- ✓ Developers often fix code smells one at a time
- ✓ In real projects, smells interact: fixing one can create another
- ✓ Current tools don't reliably handle these dependencies, so they can choose an order that introduces new smells or leaves the code only partially improved.
- ✓ LLMs can edit code, but the order of edits still matters
- ✓ Goal: optimize refactoring process by planning a safer order of edits

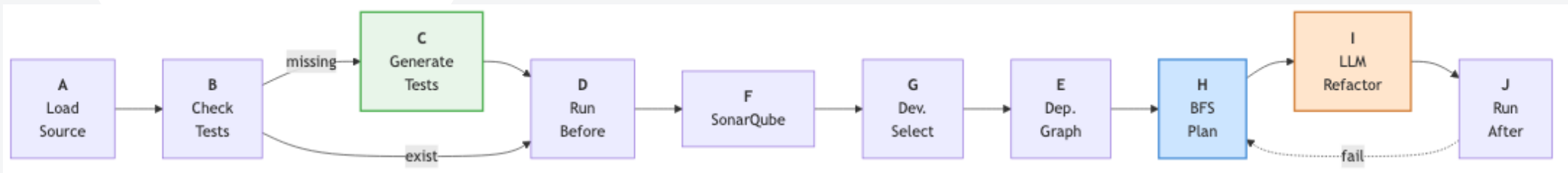
Dependency model



- ✓ Each detected smell is a node in a graph.
- ✓ Green edges show cases where one refactoring may remove another smell.
- ✓ Red edges show cases where one refactoring may create another smell.
- ✓ The planner uses this graph to choose a safer order of actions.
- ✓ The graph can also store extra attributes, for example smell weight or project-specific priority.

Code processing pipeline

1. Load the Java project and check the build/test setup.
2. *Detect* smells with SonarQube.
3. Let the developer choose target smells.
4. Build the smell dependency graph.
5. Plan the order of refactorings.
6. Apply changes with an LLM agent and validate them with tests and code quality metrics.



Planning algorithms

Refactoring order matters: one fix can remove or create other smells.

- ✓ The planner builds a dependency graph between detected smells
- ✓ Green edges show smells that may disappear together.
- ✓ Red edges show smells that may be introduced by a refactoring.

We propose two approaches:

- ✓ Greedy planning selects the best-looking smell now, but may trigger new problems
- ✓ Best-First Search looks ahead and chooses the next step that leads to a cleaner future state.

Future work

- ✓ Evaluate planning quality across developer-committed refactoring sequences.
- ✓ Validate dependency rules against empirical smell trajectories.
- ✓ Extend the dependency model with additional smell types and refactoring operations.
- ✓ Measure plan efficiency, negative dependency rate, and compile/test pass rate after each LLM patch.

InnovAite
SLOVAKIA

Thank You for Your Attention



Financované
Európskou úniou
NextGenerationEU

PLÁN [OBNOVY]



ÚRAD PODPREDESEDU VLÁDY
SLOVENSKEJ REPUBLIKY
PRE PLÁN OBNOVY
A ZNALOSTNÚ EKONOMIKU



VÝSKUMNÁ
A INOVAČNÁ
AUTORITA